

UNIVERSIDAD PRIVADA ANTENOR ORREGO



"Desarrollo e implementación de un robot móvil multifuncional reconfigurable mecánicamente para adaptarse a fundos agrícolas con diferentes camellones y entre surcos variables de la Región La Libertad-Perú"

PE5010-86701-2024-PROCIENCIA

DOCUMENTACIÓN TÉCNICA Y EVALUACIÓN DE ALGORITMOS DE IA

SISTEMA DE DETECCIÓN, CONTEO Y CLASIFICACIÓN DE FRUTOS

INFORME TECNICO 4

Autor:

Percy Brayam Cubas Muñoz

Fecha:

5 de mayo de 2026

Trujillo - Perú

MAYO - 2026

ÍNDICE

INTRODUCCIÓN

El proyecto PE5010-86701-2024-PROCIENCIA desarrolla un robot móvil multifuncional para fundos agrícolas de la Región La Libertad. La plataforma de software, denominada Robot Platform, se ejecuta sobre la computadora embebida NVIDIA Jetson Xavier y permite al operador detectar y contar frutos en tiempo real.

El desarrollo avanza en dos frentes complementarios. El primero cubre la plataforma de software: arquitectura, *workers*, comunicación entre procesos y despliegue. El segundo cubre la evaluación de la estrategia de detección y conteo de frutos, con el entrenamiento y comparación de cinco familias de modelos (YOLOv8, YOLOv9, YOLOv10, YOLOv11 y YOLOv26) y dos estrategias de conteo, medidas por el error de conteo sobre videos de campo del fundo Danper.

El informe técnico previo de enero 2025 reportó la evaluación cuantitativa de los tres modelos sobre el *dataset* de arándanos y alcanzó un mAP@0,5 máximo de 0,8407 con YOLOv9 en su variante Compact a 200 épocas (Cubas, 2025). El informe técnico de abril 2026 reportó la versión inicial de la plataforma, con un único proceso monolítico para captura, inferencia y grabación; esa integración reveló problemas de aislamiento de fallos, acumulación de *frames* por desacoplamiento de tasas y conflictos de versiones de Python entre los componentes (Cubas, 2026a).

El presente informe consolida los avances posteriores a esos dos entregables. La plataforma se rediseñó hacia una arquitectura por procesos independientes que se comunican por sockets Unix, se incorporó la aceleración con TensorRT FP16 sobre los Tensor Cores de la Jetson, y se cargó YOLOv11 como modelo de validación integral de la plataforma. Se desplegó además el servidor central del laboratorio con acceso remoto seguro, se incorporó el soporte de cámara IP por RTSP, el transporte de video por WebCodecs sobre WebSocket y el registro de detecciones por *frame* para reproducir cada sesión grabada. La integración del modelo de producción (YOLOv9s, seleccionado por su menor error de conteo en el *benchmark* de campo) se encuentra en curso y se aborda en el capítulo 3.

La finalidad del informe es documentar el estado actual del sistema, presentar la evaluación cuantitativa de los modelos de detección y fundamentar las decisiones técnicas adoptadas. La clasificación de frutos por estado de madurez es una capacidad prevista del sistema, pero queda fuera del alcance de este informe y se abordará en un entregable posterior. El código fuente de la plataforma está disponible en el repositorio público <https://github.com/pqbas/robot-platform>.

OBJETIVO GENERAL

Documentar el estado actual de la plataforma Robot Platform y la evaluación de la estrategia de detección y conteo de frutos sobre videos de campo, fundamentando las decisiones técnicas que sustentan el sistema desplegado en el robot móvil.

CAPÍTULO 1

PLATAFORMA

1.1 Visión general

La *Robot Platform* es el componente de software del robot móvil, le permite ejecutar la detección, conteo y clasificación frutos mientras el robot recorre los camellones de un fundo. Además le permite al operador interactuar con el sistema mediante una interfaz web desde un celular, una tablet o una laptop conectado a la red WiFi del robot como se observa en la Figura 1.

El sistema *Robot Platform* se despliega en dos modos:

- **Modo robot:** se ejecuta en la computadora embebida del robot (NVIDIA Jetson Xavier) y se encarga de la captura de video, la inferencia con YOLO, la grabación y la clasificación de frutos.
- **Modo servidor:** se ejecuta en una PC del laboratorio, recibe los datos sincronizados desde múltiples robots y administra los modelos YOLO desplegados en cada uno.

La Figura 2 ilustra ambos modos en operación, donde cada robot envía periódicamente por HTTP los registros de sus sesiones de conteo mientras el servidor almacena los resultados de toda la flota.

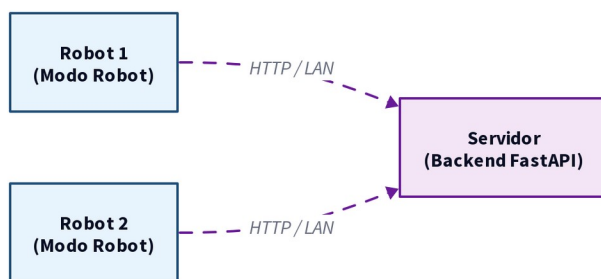


FIGURA 2. Comunicación entre el servidor central y los robots.

La Tabla 1 contrasta las funciones activas en cada modo.

Robot (Jetson Xavier)	Servidor (PC del laboratorio)
Puerto 8080	Puerto 9090
SQLite (aiosqlite)	PostgreSQL (psycopg async)
Captura de video, inferencia, grabación, conversión TensorRT	Autenticación JWT con roles
Streaming de video en tiempo real	Administración de modelos, usuarios y dispositivos
Sync push (envío de datos) y sync pull (descarga de modelos)	Recepción de sincronización y distribución de modelos
Sin autenticación (red local aislada)	Login con usuario y contraseña

TABLA 1. Funciones activas por modo de operación.

1.2 Modo robot

El modo robot se ejecuta sobre la computadora embebida (Jetson Xavier) y concentra la captura de video, la inferencia, la grabación y la conversión de modelos, según la arquitectura de la Figura 3.

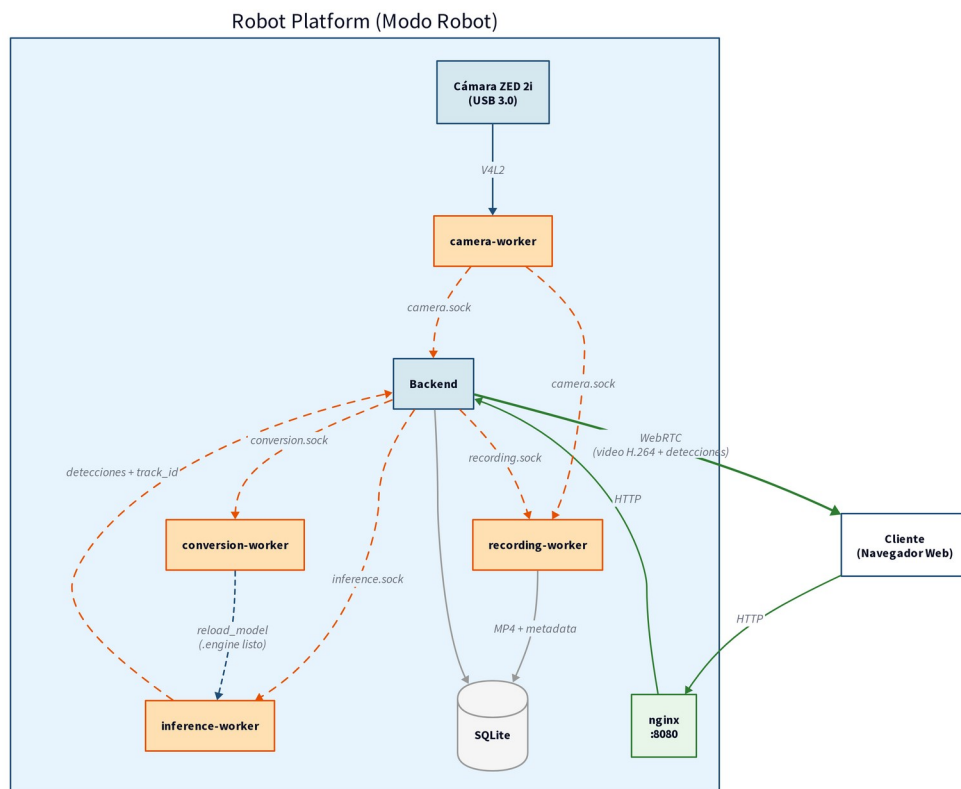


FIGURA 3. Arquitectura del sistema en modo robot.

1.2.1 Arquitectura

La arquitectura en modo robot reparte el trabajo en cinco procesos que corren en paralelo sobre la computadora embebida. Dos procesos principales orquestan el sistema y exponen la interfaz de usuario, mientras que cuatro *workers* especializados resuelven cada tarea de cómputo pesado.

Sobre la computadora embebida. Los dos procesos principales son:

- *backend* coordina el sistema y expone la API.
- *frontend* es la aplicación con la que interactúa el usuario.

Además existen 4 procesos (*workers*) que no se comunican entre sí pero tienen responsabilidades específicas especializadas:

- *camera worker* captura el video.
- *inference worker* corre la detección YOLO sobre la GPU en PyTorch o TensorRT.

- *recording worker* graba con NVENC bajo demanda.
- *conversion worker* construye *engines* TensorRT cuando el operador lo activa.

El cliente accede a través de NGINX y recibe el video en tiempo real por WebCodecs sobre WebSocket, mientras que la sincronización con el servidor central se realiza por HTTP. La Tabla 2 detalla cada proceso, así como la interfaz de comunicación, y su responsabilidad.

Proceso	Interfaz	Responsabilidad
Backend	HTTP :8080	Expone la API REST, transmite el video en tiempo real al cliente y coordina a los workers.
Frontend	navegador web	Aplicación web con la que el operador activa o detiene el conteo, observa detecciones en tiempo real, las reevisa tras la sesión y las sube al servidor
camera worker	camera.sock	Captura el video de la cámara y reparte cada frame al backend y al recording worker.
inference worker	inference.sock	Ejecuta la detección de frutos sobre la GPU con cualquier modelo YOLO y el seguimiento entre frames.
recording worker	recording.sock	Graba el video de la sesión en disco cuando el operador lo solicita.
conversion worker	conversion.sock	Acelera los modelos a TensorRT FP16 cuando el operador lo activa.

TABLA 2. Procesos del sistema en modo robot.

1.2.2 Sesión de conteo

La sesión de conteo es el flujo central del modo robot, que coordina a los *workers* para contar frutos en tiempo real. Recorre los siguientes pasos:

1. El operador inicia la sesión desde el *frontend*.
2. El *backend* marca la sesión y ordena automáticamente al *recording worker* que empiece a grabar el *stream*.
3. Durante la sesión, el *backend* envía los *frames* al *inference worker* y reenvía las detecciones al *frontend* solo como visualización en tiempo real, sin contar todavía.
4. Al finalizar, el *backend* ordena al *recording worker* el cierre y encola el conteo diferido del MP4 resultante.
5. El *counting worker* reprocesa el video (detección, seguimiento y cruce de línea), produce el conteo total y un *sidecar* por *frame*, que el *backend* guarda asociado a la grabación y a la sesión.

1.3 Modo servidor

El modo servidor se ejecuta en una PC del laboratorio, donde consolida los datos sincronizados desde múltiples robots y administra los modelos asignados a cada uno.

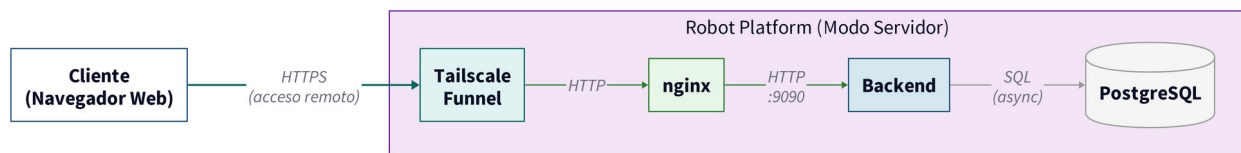


FIGURA 4. Arquitectura del sistema en modo servidor.

1.3.1 Arquitectura

Según presenta la Figura 4 y detalla la Tabla 3. En el *modo servidor* el sistema ejecuta únicamente el *backend* sin *workers* de captura ni inferencia, exponiendo una API HTTP publicada mediante Tailscale, accesible desde cualquier dispositivo que tenga el enlace y las credenciales.

Componente	Interfaz	Responsabilidad
Backend	HTTP :9090	Expone la API REST, gestiona autenticación y administración de usuarios, modelos y dispositivos, sincronización de los robots.
PostgreSQL	TCP :5432	Almacena usuarios, dispositivos, modelos y los registros que llegan de cada robot.
nginx	HTTP :80	Enruta las peticiones del cliente al backend y entrega los archivos del frontend.
Frontend	estático	Aplicación web React que nginx entrega al navegador del cliente.
Tailscale	Funnel (HTTPS)	Da acceso remoto seguro al servicio por HTTPS.

TABLA 3. Componentes del sistema en modo servidor.

1.3.2 Seguridad y control de acceso

Las credenciales son aplicadas de manera diferente según el tipo de cliente (persona, robot) de la siguiente manera:

- Las personas inician sesión con usuario y contraseña, y reciben un token JWT
- Los robots se autentican con una clave de dispositivo

Además se han aplicado las siguientes 5 medidas para reforzar el acceso:

- limitar el *login* a cinco intentos cada cinco minutos
- bloquear la cuenta treinta minutos tras cinco fallos en quince minutos

- restringir el CORS a la URL pública
- añadir cabeceras de seguridad HTTP
- deshabilitar la documentación interactiva de la API.

1.4 Justificación del diseño por procesos

La descomposición del sistema en más cuatro *workers* responde a problemas concretos que surgieron al integrar el sistema monolítico anterior.

- **Aislamiento de fallos:** anteriormente un fallo del modelo o de la cámara dejaba el *backend* irrecuperable, por lo que se optó que cada componente tenga un proceso independiente, en consecuencia un fallo aislado ya no interrumpe el todo el sistema.
- **Desacoplamiento de tasas de *frame*:** la captura opera entre 20 y 30 FPS y la inferencia entre 9 y 14 FPS, por lo que en un mismo proceso los *frames* se acumulaban y retardaban el video, al separar cada *worker* por responsabilidad cada uno avanza a su ritmo sin acumulación.
- **Acceso exclusivo a la cámara:** el driver de la computadora embebida no admite varios consumidores sobre una misma cámara, lo que impedía integrar la grabación de video, por lo que el *camera worker* centraliza la captura y reparte los *frames* por colas al *backend* y al *recording worker*.
- **Independencia de versiones:** el *backend* y los *workers* usan librerías específicas que pueden no ser compatibles entre sí, sobre todo las de Deep Learning, ya que la computadora embebida trae drivers atados a una versión concreta de PyTorch que exige una versión de Python en conflicto con la del *backend*, por lo que cada componente corre en su propio entorno aislado.
- **Recursos liberados en reposo:** mantener la cámara y los modelos cargados de forma permanente ocupaba NVENC, GPU y memoria entre sesiones, por lo que el *recording worker* y el *conversion worker* no abren la cámara ni cargan modelos hasta recibir un comando del *backend*.
- **Recarga de modelo en caliente:** cambiar de modelo exigía reiniciar el proceso de inferencia, por lo que el *inference worker* acepta una orden para cargar un nuevo modelo, en PyTorch o TensorRT, sin reiniciarse ni interrumpir el servicio.
- **Monitoreo independiente:** un solo journal mezclaba los logs de todos los componentes, por lo que cada proceso es una unidad systemd separada con su propio log y puede depurarse sin afectar al resto.

1.5 Descripción del *backend*

El *backend* es un servicio FastAPI que actúa como coordinador central del sistema, con un único codebase que se comporta de forma distinta según el modo (*modo robot*, *modo servidor*)

En *modo robot* orquesta a los *workers*, expone la API y transmite el video en tiempo real, mientras que en *modo servidor* consolida los datos sincronizados de varios robots sobre PostgreSQL y administra los modelos.

1.5.1 Modo Robot

En *modo robot* el *backend* expone un conjunto de operaciones por su API REST (vease la Tabla 4), además controla cada *worker* (*camera worker*, *inference worker*, *recording worker*, *conversion worker*, *counting worker*) enviando comandos puntuales sobre su socket Unix (vease la Tabla 5).

Dominio	Operación que permite	Endpoint
Streaming de video	Entrega el <i>stream</i> en vivo por WebRTC, WebCodecs sobre WebSocket o MJPEG.	POST /offer, WS /ws/stream
Conteo y sesiones	Inicia, cierra y elimina sesiones de conteo y registra los eventos de cruce.	POST /api/counting/start, GET /api/sessions
Configuración	Ajusta el modo de conteo, el umbral, la dirección y la línea o ROI.	PUT /api/config/counting
Modelos	Activa la aceleración TensorRT por modelo.	PUT /api/models/{uuid}/tensorrt
Grabaciones	Genera y sirve los archivos MP4 y su registro de detecciones por <i>frame</i> .	GET /api/recordings/{uuid}/file
Sincronización	Envía el <i>push</i> de sesiones, eventos y modelos al servidor central.	POST /api/sync/push

TABLA 4. Operaciones que expone el backend en modo robot.

Worker	Comando	Función
camera worker	reload	Recarga el preset y reabre la cámara con la nueva resolución y FPS.
	status	Devuelve la resolución y los FPS actuales.
inference worker	reload_model	Carga un nuevo modelo YOLO desde model_path, con filtro opcional de clases.
	status	Devuelve la ruta del modelo activo.
	timing	Devuelve las estadísticas de tiempo de inferencia.
recording worker	start	Inicia la grabación MP4 tomando los <i>frames</i> del socket de cámara.
	stop	Finaliza la grabación y devuelve su duración y tamaño.
	status	Devuelve el estado de grabación, inactivo o grabando.
conversion worker	convert	Encola la construcción de un engine TensorRT a partir de un modelo PyTorch.
	status	Devuelve el estado de conversión y el último

		resultado.
counting worker	count	Encola el reprocesamiento offline de un MP4 grabado para producir el conteo.
	status	Devuelve el estado de conteo y el último resultado.

TABLA 5. Comandos de control que admite cada worker.

1.5.2 Modo Servidor

En *modo servidor* el *backend* expone las operaciones de administración y sincronización que resume la Tabla 6.

Dominio	Operación que permite	Endpoint
Conteo y sesiones	Consulta las sesiones y eventos sincronizados desde los robots.	GET /api/sessions/{session_id}/events
Modelos	Asigna los modelos a cada robot.	PUT /api/devices/{device_id}/models
Grabaciones	Consulta las grabaciones sincronizadas.	GET /api/recordings/{uuid}/file
Sincronización	Recibe el <i>push</i> de los robots y distribuye los modelos por <i>pull</i> .	POST /api/sync/sessions, GET /api/sync/models/{model_uuid}
Administración	Autentica con JWT y gestiona usuarios, empresas y dispositivos.	POST /api/auth/login

TABLA 6. Operaciones que expone el backend en modo servidor.

1.6 Descripción de los workers

Los cinco *workers* (*camera worker*, *inference worker*, *recording worker*, *conversion worker* y *counting worker*) son independientes del backend, cada uno con su propio directorio y entorno virtual, a continuación se detalla el funcionamiento y responsabilidades de cada *worker*.

1.6.2 camera worker

El *camera worker* centraliza la captura de video y ofrece tres capacidades:

- Reparte cada *frame* a varios consumidores desde una sola conexión a la cámara
- Permite cambiar la resolución con el sistema en marcha
- Soporta cámaras USB como IP.

La primera capacidad es la más importante, ya que evita abrir la cámara más de una vez y permite que el *backend* y el *recording worker* operen en simultáneo sobre la misma fuente.

Para lograrlo, el *camera worker* aplica un esquema de *fan-out* con colas independientes, como ilustra la Figura 5, que sigue dos pasos:

1. Abre la cámara una sola vez por V4L2 y entrega cada *frame* a una cola independiente por consumidor (*cola backend*, *cola recording*).
2. Si un *worker* se atrasa, descarta el *frame* más antiguo de su cola y conserva el más reciente, de modo que un *worker* lento no frena a la captura ni a los demás y cada uno avanza siempre sobre el *frame* vigente.

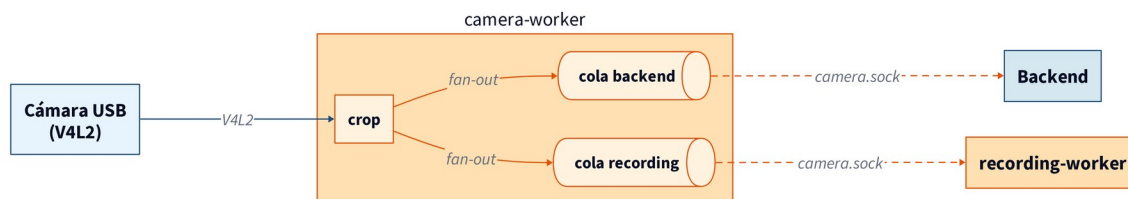


FIGURA 5. Esquema de *fan-out* del *camera worker* con colas *drop-oldest* por consumidor.

1.6.3 inference worker

El *inference worker* ejecuta la detección sobre cada *frame* que llega del *backend* y ofrece dos capacidades:

- Ejecuta cualquier modelo YOLO sobre la GPU (tanto en PyTorch como en TensorRT)
- Permite recargar el modelo activo con el sistema en marcha, sin reiniciar el proceso

La primera capacidad es la que permite detectar en tiempo real, ya que tanto la selección de modelos (familia YOLO) como las optimizaciones (TensorRT) mantienen el tiempo de inferencia en el orden de los milisegundos.

Además, el *backend* puede ordenar la recarga del modelo activo sin reiniciar el proceso, lo que se aplica tras una sincronización con el servidor o tras una conversión TensorRT recién terminada.

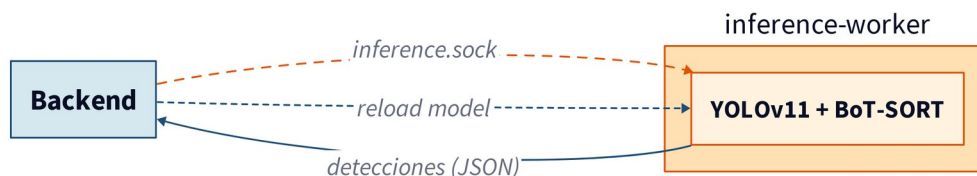


FIGURA 6. Secuencia del *inference worker*: intercambio por *frame* y recarga en caliente entre el *backend* y el *inference worker*.

La Figura 6 resume el intercambio entre el *backend* y el *inference worker*, que sigue dos flujos:

1. Por cada *frame*, el *backend* envía la imagen JPEG sobre el socket *inference.sock* y recibe las detecciones.
2. En la recarga en caliente, el *backend* ordena cargar un nuevo modelo, en PyTorch o TensorRT, sin reiniciar el proceso ni interrumpir el *streaming*.

1.6.4 recording worker

La función principal del *recording worker* es grabar un MP4 por sesión, sus capacidades son:

- Permanece en reposo hasta recibir la orden de inicio
- Selecciona el codificador según la plataforma y autoescala el bitrate según la altura del *frame*
- Guarda un registro de detecciones por *frame* enlazado al mismo video

La primera capacidad evita que el *worker* reserve NVENC, CPU y la conexión con la cámara mientras no se graba, por lo que esos recursos quedan libres para el *streaming* y la inferencia.

La segunda aprovecha el codificador por hardware de la Jetson (nvv4l2h264enc) y de escritorio NVIDIA (h264_nvenc), y solo cae a libx264 por software cuando no hay GPU disponible, como resume la Tabla 7.

La tercera permite reproducir la sesión con las detecciones superpuestas y sincronizadas, y es lo que sirve la operación de grabaciones de la Tabla 4.

Plataforma	Codificador	Bitrate (1080p / 720p)
Jetson Xavier (GStreamer)	nvv4l2h264enc	12 / 8 Mbps
Desktop NVIDIA (PyAV)	h264_nvenc	12 / 8 Mbps
Sin GPU (PyAV fallback)	libx264	9 / 6 Mbps

TABLA 7. Backends de codificación seleccionados por el *recording worker*.

La Figura 7 resume el flujo de grabación entre el *backend*, el *camera worker* y el *recording worker*, que sigue tres pasos:

1. El *backend* ordena start y stop sobre el socket *recording.sock*.
2. El *worker* recibe los *frames* del *camera worker* por el socket *camera.sock* y los codifica a H.264 con el codificador por hardware.
3. El *worker* emite el MP4 fragmentado a disco.

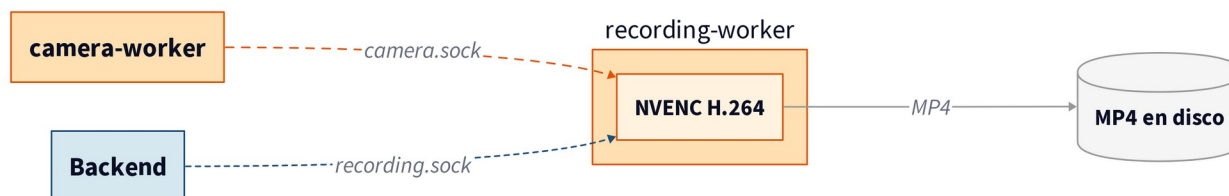


FIGURA 7. Flujo del *recording worker* entre el *backend*, el *camera worker* y el *recording worker*.

1.6.5 conversion worker

La función principal del *conversion worker* es construir *engines* TensorRT FP16 a partir de modelos YOLO en PyTorch. Sus capacidades son:

- Atiende solicitudes sobre el socket *conversion.sock* y procesa una conversión a la vez, rechazando con 409 si llega otra mientras hay una en curso
- Cachea cada *engine* con el sha256 del modelo PyTorch de origen incrustado en el nombre del archivo, lo que invalida la cache cuando el modelo se reentrena

La conversión a TensorRT FP16 permite que la Jetson Xavier ejecute el proceso de detección de manera acelerada, ya que TensorRT aprovecha operaciones de matriz en FP16 PyTorch no utiliza, por lo que TensorRT reduce la latencia por *frame* y eleva el FPS efectivo.

La Tabla 8 resume la latencia de inferencia aislada (percentiles p50 y p99 sobre 600 *frames* a 640x640 con `sudo jetson_clocks`) y el FPS efectivo medido de extremo a extremo sobre el flujo de producción del robot.

Backend de inferencia	Latencia p50	Latencia p99	FPS efectivo
PyTorch FP32 sobre CUDA	~75 ms	~85 ms	9
TensorRT FP16	50,9 ms	57,0 ms	14

TABLA 8. Rendimiento de inferencia YOLO sobre Jetson Xavier (latencia de inferencia aislada y FPS efectivo medido de extremo a extremo).

En la Jetson, el *conversion worker* se ejecuta sobre el Python del sistema para reutilizar los bindings de TensorRT que provee JetPack. La Figura 8 detalla el ciclo completo, desde que se activa TensorRT hasta la recarga en caliente del *engine*, en tres pasos:

1. El *backend* encola una conversión sobre el socket *conversion.sock*.
2. El *worker* construye el *engine* FP16, una conversión a la vez y con respuesta 409 si hay otra en curso, mientras el *backend* sondea el estado.
3. Al terminar, el *backend* ordena al *inference worker* recargar el *engine* en caliente.

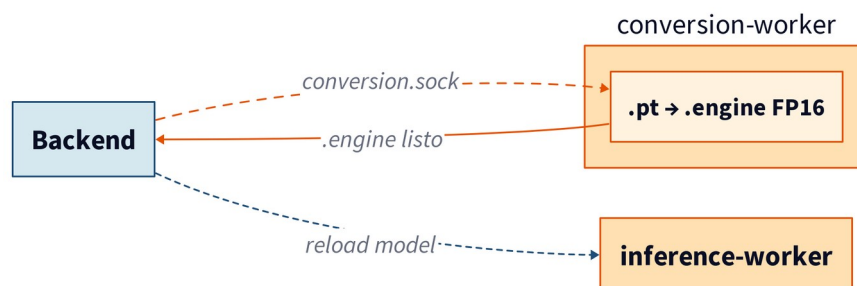


FIGURA 8. Secuencia del *conversion worker*, desde la activación de TensorRT hasta la recarga en caliente del *engine*.

1.6.6 counting worker

El conteo en vivo presenta varios inconvenientes de desfase debido a la latencia variable de los diferentes componentes, por esta razón se ha creado un worker dedicado al conteo post-session.

El *worker* permanece en reposo, sin hilo ni GPU, hasta recibir un trabajo, y atiende una sola tarea a la vez de modo que una segunda solicitud recibe busy. Por cada trabajo decodifica el MP4 *frame* a *frame*, ejecuta detección con el mismo *engine* que fijó la sesión, sigue los objetos con ByteTrack y cuenta los cruces de la línea de referencia, emitiendo dos salidas:

1. El conteo total acumulado de la sesión, que el *backend* persiste como resultado definitivo.
2. Un *sidecar* {uuid}.jsonl con una línea por *frame* que registra las detecciones, su identificador de seguimiento y el conteo acumulado, alineado con el video por su marca de presentación para reproducir luego la sesión con las detecciones superpuestas.

El *counting worker* admite dos métodos por objeto:

- *single*: un detector sobre la región de interés, el comportamiento histórico
- *tilled*: dos mosaicos cuadrados de lado H/2 apilados y centrados, cada uno con su propio detector, seguimiento y línea de cruce, cuyos conteos se suman

La Figura 9 resume el flujo, que el *backend* dispara al cerrar una sesión de conteo o al reprocesar una grabación con un modelo fijado, y cuyo estado sondea hasta que termina.

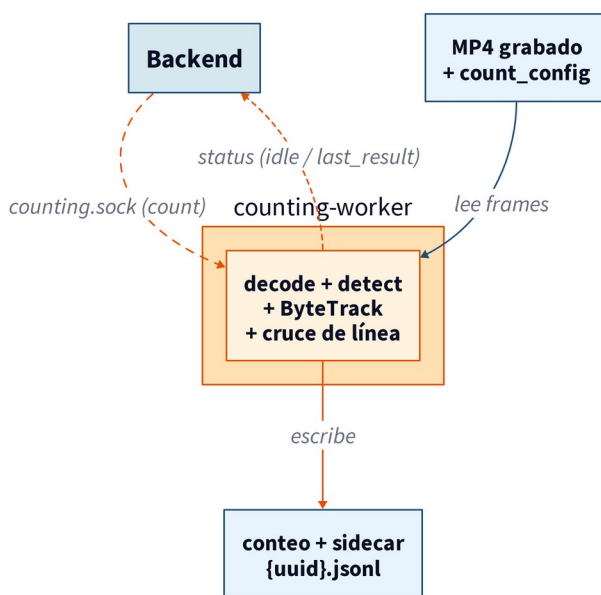


FIGURA 9. Flujo del counting worker: reprocesa el MP4 grabado para producir el conteo definitivo y el sidecar {uuid}.jsonl alineado por frame.

1.7 Descripción del *frontend*

El *frontend* es una aplicación de página única (SPA) construida con React 19 y TypeScript sobre Vite, estilizada con Tailwind CSS y componentes shadcn/ui. Sus características son:

- Se compila a archivos estáticos que nginx entrega al navegador
- Maneja el estado global con la Context API de React, repartido en un contexto que conserva la sesión autenticada y otro que expone el modo de operación (ROBOT_MODE)
- Separa la comunicación con el *backend* en un cliente HTTP único que adjunta el token a cada llamada REST y un canal de video independiente

La comunicación pasa siempre por nginx, que entrega los archivos estáticos y actúa de *proxy* inverso hacia el *backend*. La Figura 10 resume esta conexión, con el cliente HTTP que transporta datos, configuración y sesiones por REST y la capa de video sobre un canal WebSocket o WebRTC separado.

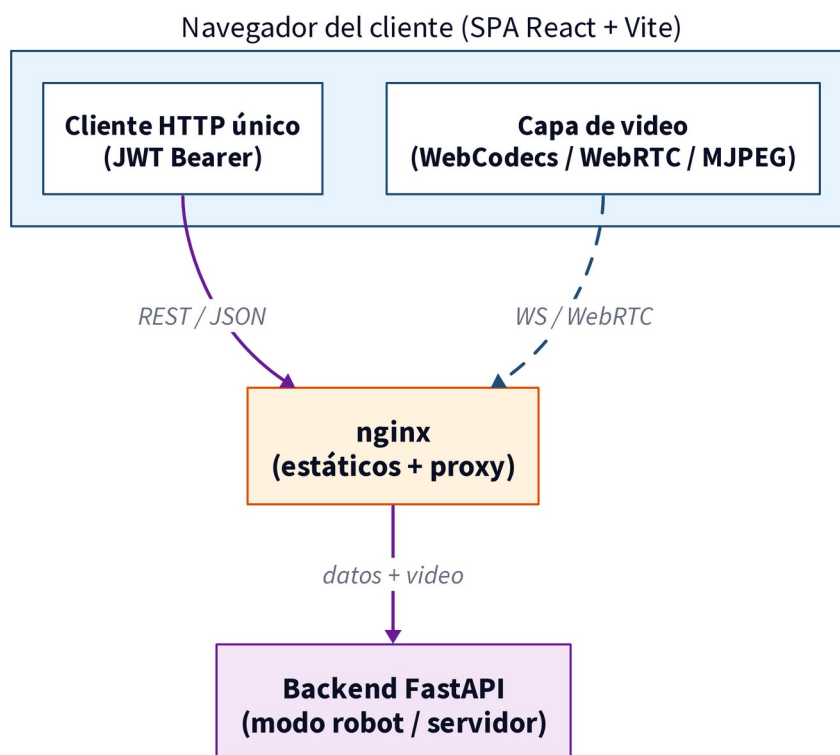


FIGURA 10. Conexión entre el frontend y el backend: el cliente HTTP único transporta los datos por REST y la capa de video usa un canal WebSocket o WebRTC separado, ambos a través de nginx.

1.7.2 Adaptación por modo y rol

Una misma base de código sirve los dos despliegues, y las vistas disponibles junto con la protección de las rutas se deciden en tiempo de ejecución según el valor de ROBOT_MODE (*modo robot*, *modo servidor*) y el rol declarado en el JWT del usuario.

- En modo robot el acceso es directo, orientado al operador en campo, y se habilitan los módulos de visión, sesiones, mapa, grabaciones y configuración.
- En modo servidor el ingreso exige autenticación y se ofrecen sesiones, grabaciones, configuración y el tablero analítico, con las páginas de administración reservadas a los usuarios con rol administrador.
- El acceso a cada vista está protegido según la sesión y el rol del usuario, de modo que las páginas restringidas no se cargan aunque se ingrese su URL directa.

1.7.3 Transporte de video y resiliencia

El video en vivo es independiente del canal de datos y admite tres transportes seleccionables, con WebCodecs sobre WebSocket como opción por defecto:

- WebCodecs sobre WebSocket aprovecha el decodificador H.264 por hardware del dispositivo cliente y descarta cuadros P cuando la cola se acumula.
- WebRTC negocia la conexión por SDP y recibe las detecciones por un canal de datos.
- MJPEG sobre WebSocket transmite cuadros JPEG con control de flujo por crédito.

Cada transporte incorpora reconexión automática mediante un detector de congelamiento que reinicia la conexión cuando deja de llegar video, con reintentos espaciados de forma creciente antes de declarar la conexión fallida.

CAPÍTULO 2

GUÍA DE USO

2.1. Configuración de su navegador

1. Abrir un navegador Chrome
2. Colocar la siguiente "expresion" en su barra "chrome://flags" (ver FIGURA 11)
3. Buscar la configuracion "Insecure origins treated as secure", y configurar las siguientes direcciones IP de los robots moviles, Robot 1 `http://192.168.50.103`, Robot 2 `http://192.168.50.113` (ver FIGURA 12)

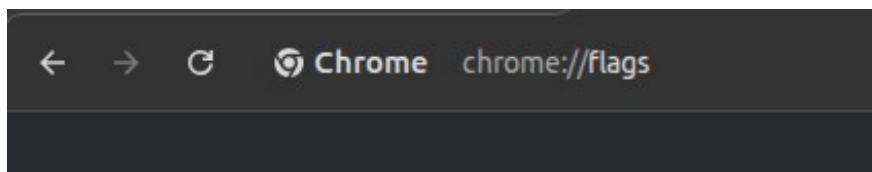


FIGURA 11. Página `chrome://flags` del navegador Chrome, desde donde se habilitan los orígenes inseguros tratados como seguros.



FIGURA 12. Opción "Insecure origins treated as secure" con las direcciones IP de los robots móviles agregadas.

2.2. Ingreso a la plataforma

1. Conectarse al internet del laboratorio (LABINM ROBOTICA)
2. Conectar cada robot al internet del laboratorio (LABINM ROBOTICA)
3. Abrir un navegador Chrome (ver FIGURA 13)
4. Colocar la IP en la red LABINM ROBOTICA correspondiente al robot de interes, Robot 1 `192.168.50.113`, Robot 2 `192.168.50.103` (ver FIGURA 14)
5. Se abra automaticamente la siguiente interfaz (ver FIGURA 15)

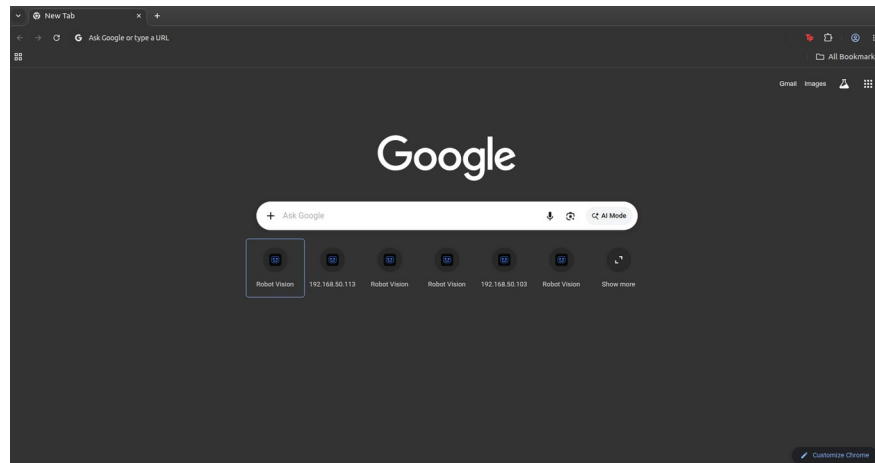


FIGURA 13. Navegador Chrome abierto, listo para ingresar la dirección del robot.

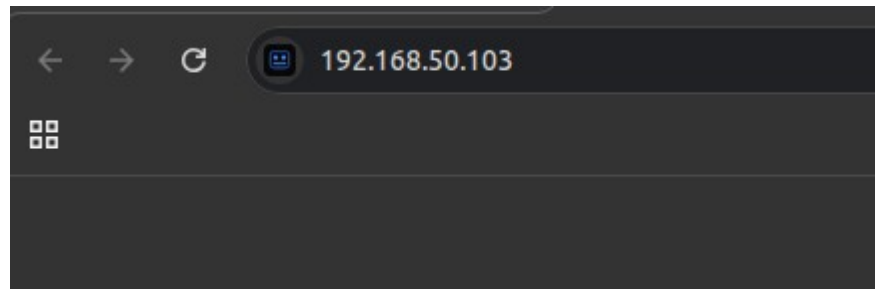


FIGURA 14. Barra de direcciones con la IP del robot en la red LABINM ROBOTICA.

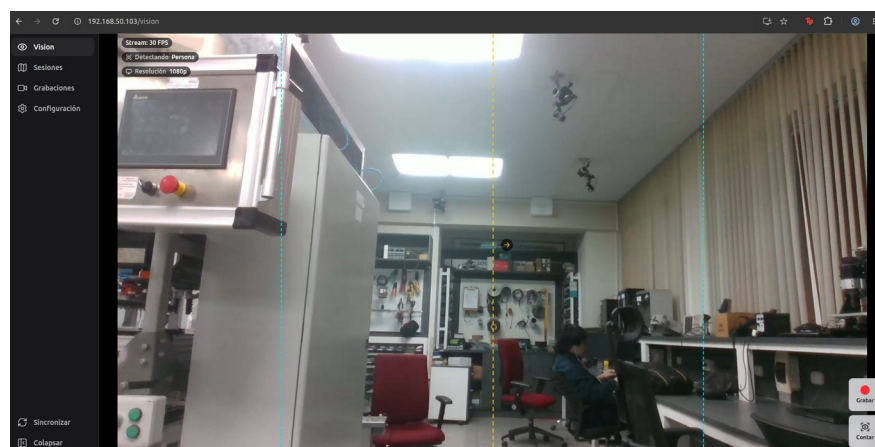


FIGURA 15. Interfaz de la plataforma que se abre automáticamente tras ingresar la dirección del robot.

2.3. Ejecución del conteo

1. Ingresar a la plataforma del robot móvil, seleccionar el menú de "Visión" y verificar que el sistema obtiene la imagen de la cámara (ver FIGURA 16).

2. El panel de estado, en la esquina superior izquierda, muestra los FPS del stream, el modelo activo, la clase objetivo y la resolución (ver FIGURA 17).
3. Presionar el botón "Contar", señalado por la flecha, para iniciar la sesión de conteo (ver FIGURA 18).
4. Presionar el botón "Detener", señalado por la flecha, para finalizar la sesión de conteo (ver FIGURA 19).
5. Al finalizar, la sesión queda registrada como una nueva entrada en el menú de sesiones, descrito en la sección 2.10 (ver FIGURA 20).

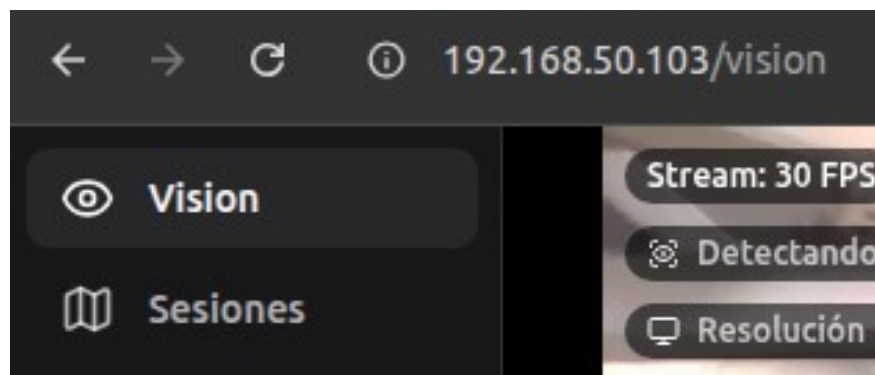


FIGURA 16. Menú de visión con la imagen en vivo de la cámara del robot.

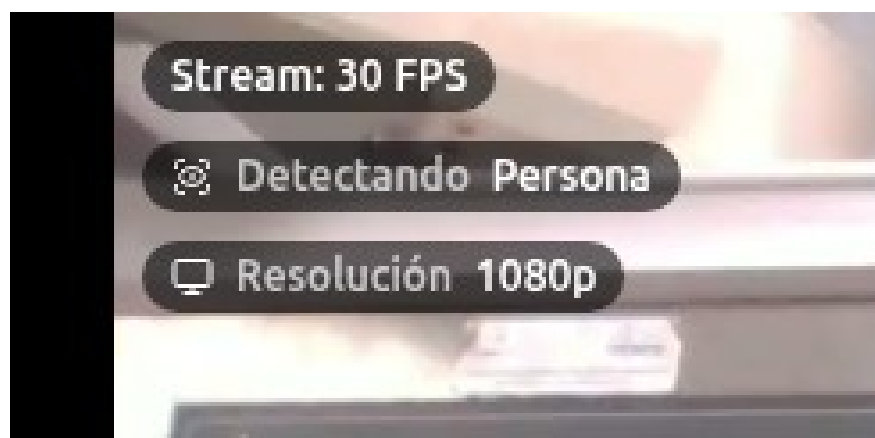


FIGURA 17. Panel de estado, en la esquina superior izquierda, con los FPS del stream, el modelo activo, la clase objetivo y la resolución.

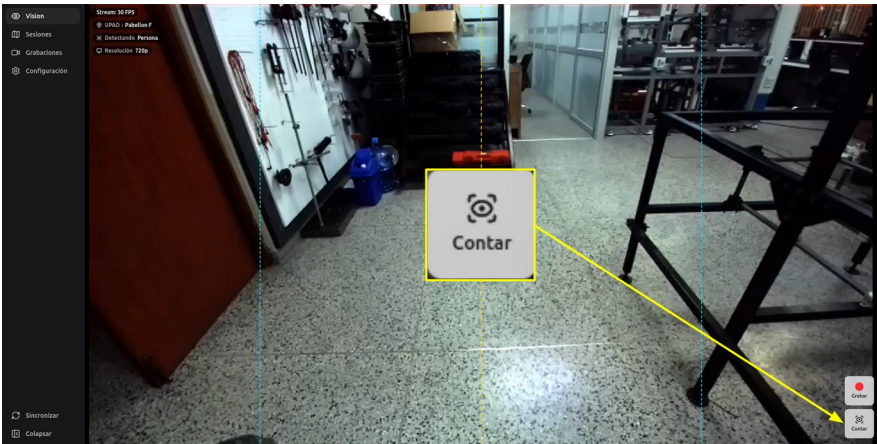


FIGURA 18. Módulo de visión en tiempo real; la flecha señala el botón "Contar" que inicia la sesión de conteo.

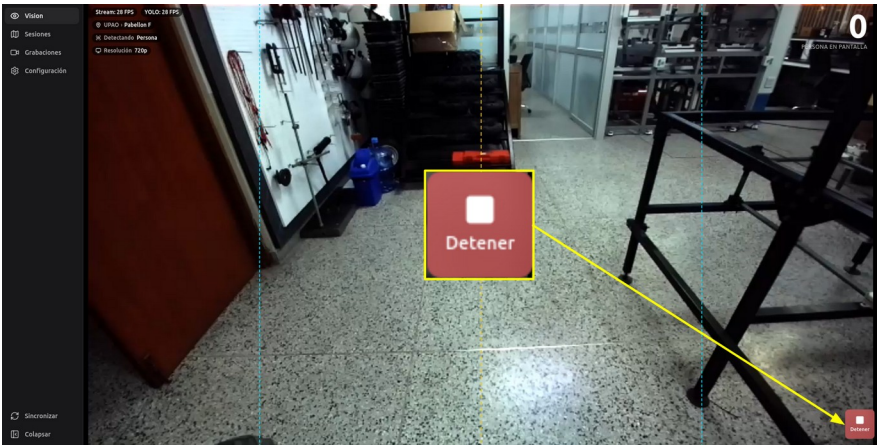


FIGURA 19. Sesión de conteo en curso; la flecha señala el botón "Detener" que finaliza la sesión.

Sesiones							
Clase		Desde		Hasta			
Todas		mm / dd / yyyy		mm / dd / yyyy			
Fecha	Clase	Conteo	Duración	Tamaño	Estado		Acciones
30 may 2026, 22:06	humano	0	0s	714 KB	Terminado		🔄 📄 🗑️ ⚙️
30 may 2026, 22:24	humano	1	0s	302 KB	Terminado		🔄 📄 🗑️ ⚙️
30 may 2026, 22:08	humano	1	7s	404 KB	Terminado		🔄 📄 🗑️ ⚙️

FIGURA 20. Menú de sesiones con la sesión recién finalizada resaltada como la primera fila.

2.4. Ejecución de la grabación

1. Ingresar a la plataforma del robot móvil, seleccionar el menú de "Visión" y verificar que el sistema obtiene la imagen de la cámara.
2. Presionar el botón "Grabar", señalado por la flecha, para iniciar la grabación (ver FIGURA 21).
3. Presionar el botón "Detener", señalado por la flecha, para finalizar la grabación (ver FIGURA 22).
4. Al finalizar, la grabación queda registrada como una nueva entrada en el menú de grabaciones, descrito en la sección 2.11 (ver FIGURA 23).

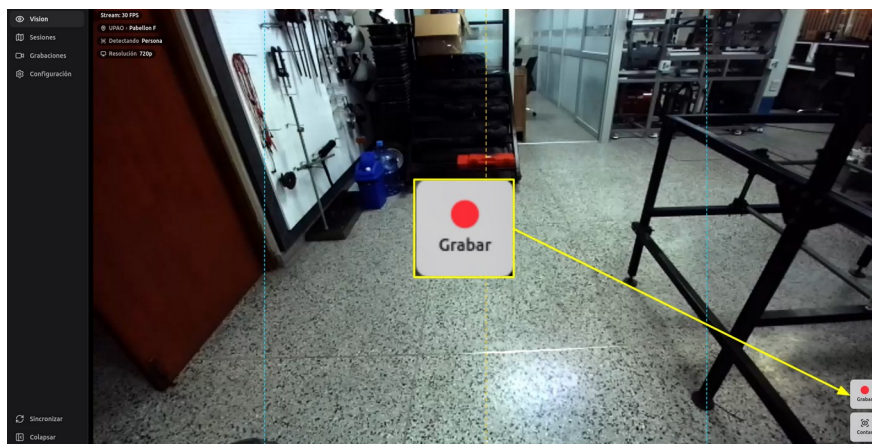


FIGURA 21. Módulo de visión en tiempo real; la flecha señala el botón "Grabar" que inicia la grabación.

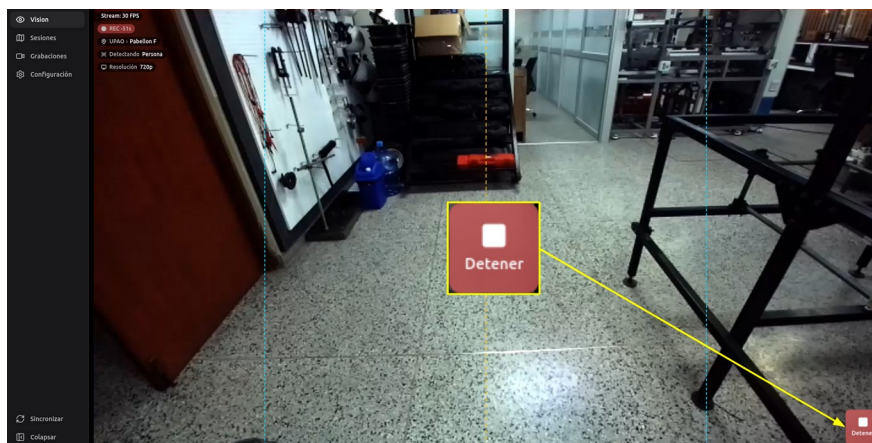


FIGURA 22. Grabación en curso; la flecha señala el botón "Detener" que finaliza la grabación.

Fecha	Duración	Tamaño	Estado	Acciones
30 may 2024, 22:25	4s	714 KB	Pendiente	
30 may 2024, 22:24	6s	382 KB	Terminado	
30 may 2024, 22:07	7s	404 KB	Terminado	

FIGURA 23. Menú de grabaciones con la grabación recién finalizada resaltada como la primera fila, en estado "pendiente".

2.5. Configuración de la cámara

- Ingresar al menú de configuración y abrir la sección de cámara (ver FIGURA 24).
- Las opciones de fuente de video disponibles son dos, la cámara USB conectada por cable y la cámara IP por red (RTSP o HTTP) (ver FIGURA 25).

Configuración

Cámara
Dispositivo de captura y resolución de la imagen

Fuente de video
Cámara USB

USB: cámara conectada por cable. IP: cámara por red (RTSP/HTTP).

Cámara

El cambio se aplica en la próxima conexión.

Resolución de captura
720p

El cambio reinicia la cámara; detén conteo y grabación antes.

Reiniciar cámara
Reiniciar cámara

Reinicia el proceso de la cámara si se queda congelada o no reconecta tras desconectar/conectar el cable. El video se corta unos segundos.

Guardar

FIGURA 24. Sección de cámara del menú de configuración del modo robot.

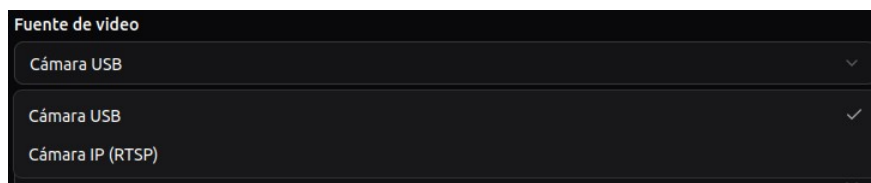


FIGURA 25. Selector de fuente de video con las opciones "Cámara USB" y "Cámara IP (RTSP)".

2.5.1 Configuración de cámara USB

- La cámara USB habilita tres opciones (ver FIGURA 26):
 - En el selector de cámara se elige cuál de los dispositivos USB conectados se usa como fuente de video.
 - En la resolución de captura se define si la cámara entrega el video en 720p o en 1080p.
 - Con el botón de reinicio se reinicia el *camera worker*, útil si la cámara se congela o no reconecta tras desconectar y conectar el cable.
- El cambio de resolución reinicia la cámara, por lo que conviene detener el conteo y la grabación antes.

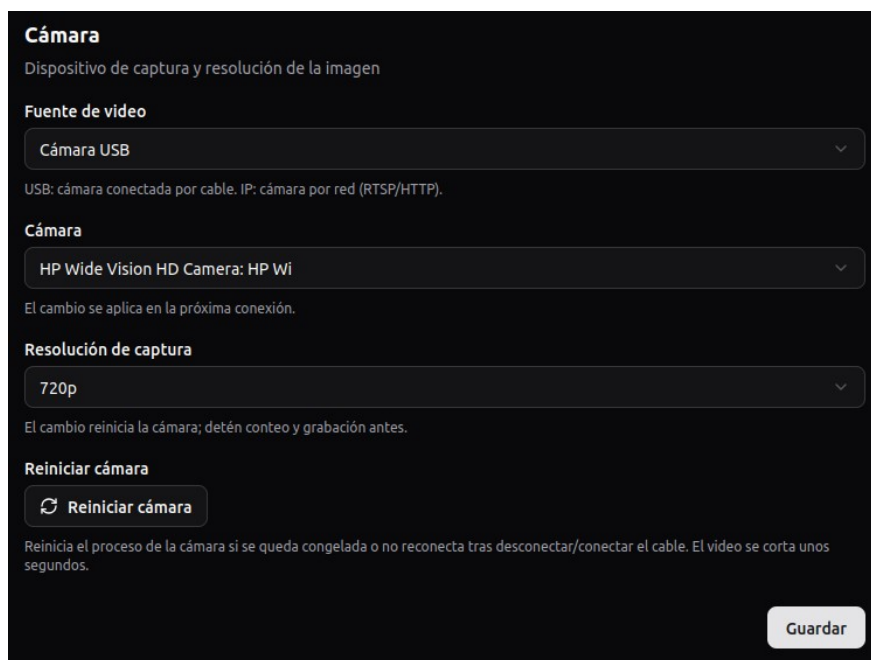
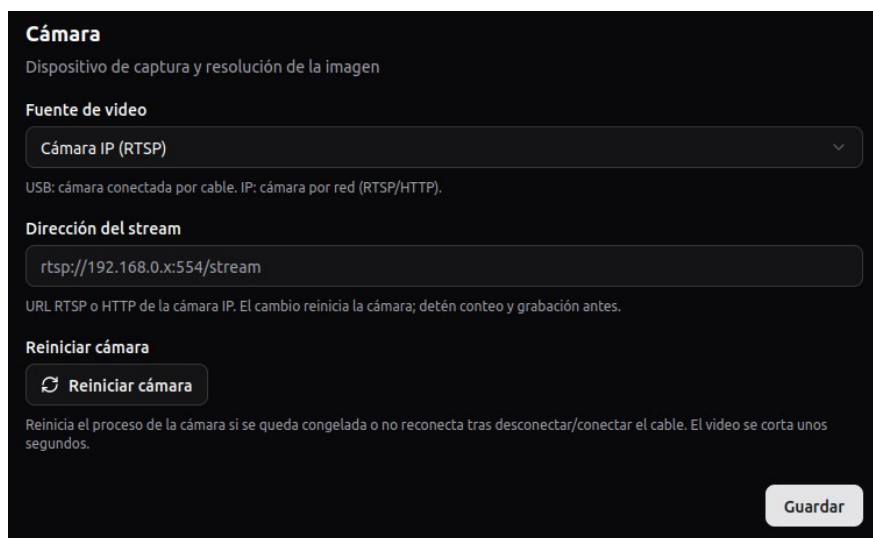


FIGURA 26. Panel de configuración de la cámara USB con el dispositivo, la resolución y el reinicio del *camera worker*.

2.5.2 Configuración de cámara IP

- La cámara IP habilita dos opciones (ver FIGURA 27):

- En la dirección del stream se ingresa la URL RTSP o HTTP que publica la cámara, de la forma `rtsp://<host>:<puerto>/stream`, desde donde el robot toma el video.
- Con el botón de reinicio se reinicia el *camera worker*, útil si el stream se congela o la cámara deja de responder.



El panel de configuración de la cámara IP tiene un fondo oscuro. En la parte superior, el título "Cámara" está en un color claro. Debajo, el subtítulo "Dispositivo de captura y resolución de la imagen" también es claro. La sección "Fuente de vídeo" contiene un menú desplegable con "Cámara IP (RTSP)" seleccionado. Debajo de este, una línea de texto explica: "USB: cámara conectada por cable. IP: cámara por red (RTSP/HTTP)". La sección "Dirección del stream" tiene un campo de texto con el valor "rtsp://192.168.0.x:554/stream". Debajo de este campo, una línea de texto explica: "URL RTSP o HTTP de la cámara IP. El cambio reinicia la cámara; detén conteo y grabación antes.". La sección "Reiniciar cámara" contiene un botón con un ícono de recarga y el texto "Reiniciar cámara". Debajo de este botón, una línea de texto explica: "Reinicia el proceso de la cámara si se queda congelada o no reconecta tras desconectar/conectar el cable. El video se corta unos segundos.". En la parte inferior derecha, hay un botón "Guardar".

FIGURA 27. Panel de configuración de la cámara IP con la dirección del stream y el reinicio del *camera worker*.

2.6. Configuración de la detección

- La sección de detección permite configurar tres aspectos (ver FIGURA 28):
 - En el objeto a detectar se elige el modelo y su archivo de pesos, junto con el backend de inferencia, que puede ser PyTorch o TensorRT.
 - En el área de detección se decide si YOLO procesa un cuadrado central del frame, recomendado porque evita el *letterbox*, o el frame completo.
 - En el umbral de confianza se fija el valor mínimo por debajo del cual las detecciones se descartan.
- El *backend* aplica cada cambio recargando en caliente el modelo del *inference worker*, sin interrumpir el *streaming*.



FIGURA 28. Panel de configuración de la detección con el objeto a detectar, el área de detección y el umbral de confianza.

2.7. Configuración del conteo

- La sección de conteo permite configurar cuatro aspectos:
 - En el método de conteo se elige, para cada modelo por separado, entre single, que usa una sola región de interés, y tiled, que divide el frame en dos regiones apiladas.
 - En el modo de conteo se define la orientación de la línea, horizontal con línea vertical o vertical con línea horizontal.
 - En la posición de la línea de cruce se indica dónde se ubica la línea sobre el frame, como un valor de X normalizado entre 0 y 1.
 - En la dirección de cruce se establece hacia qué lado debe cruzar un objeto para sumarse al conteo, por ejemplo de izquierda a derecha.
- El método se fija de forma independiente para cada modelo, y la línea y la dirección se aplican tanto al método single como, de forma equivalente, a cada una de las dos regiones del método tiled (ver FIGURA 29).
- Al desplegar el selector del método de conteo se muestran las dos opciones disponibles para el modelo, single y tiled, con un check sobre la opción activa (ver FIGURA 30).

Configuración

Cámara
Dispositivo y resolución

Detección
Qué detectar y cómo

Conteo
Línea de cruce y dirección

Sincronización
Forzar sync ahora

Servidor
Conexión y credenciales

Conteo
Línea virtual que cuenta objetos al cruzarla

Método de conteo por objeto
Single (line-crossing sobre el Área de detección) o Tiled (dos cuadrados H/2 apilados y centrados, con tracker propio, mejor para arándanos). Cada cambio se guarda solo. La línea y dirección de abajo solo aplican a Single.

Blueberry best_yolov11n.pt	Single
Blueberry best_yolov10n.pt	Single
Blueberry best_yolov9s.pt	Single
Humano yolo11m.pt	Single
Persona yolo11n.pt	Single
Vaso yolo11n.pt	Single
Celular yolo11n.pt	Single

Modo de conteo
Horizontal (línea vertical)

Línea de cruce (X normalizada, 0-1)
0.5

Posición relativa de la línea sobre el frame (0 = borde inicial, 1 = borde opuesto), independiente de la resolución.

Dirección de cruce
Izquierda → Derecha

Guardar

FIGURA 29. Panel de configuración del conteo con el método por objeto, el modo, la posición de la línea de cruce y la dirección.

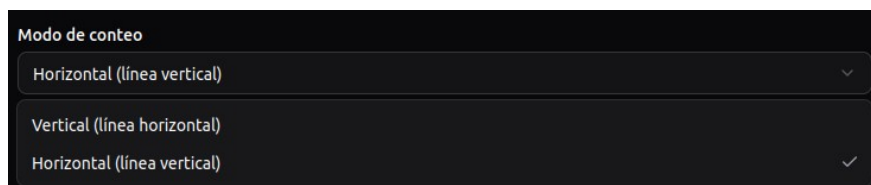
Single

Single

Tiled

FIGURA 30. Selector del método de conteo desplegado con las opciones "Single" y "Tiled".

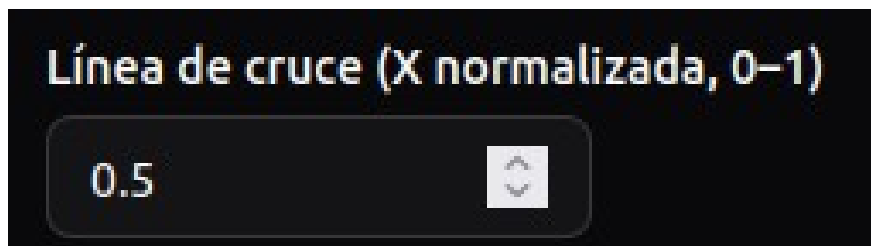
Cada control de la configuración del conteo se selecciona por separado. El modo de conteo permite elegir entre horizontal con línea vertical o vertical con línea horizontal. La línea de cruce se ajusta como un valor de X normalizado entre 0 y 1, que ubica la línea sobre el frame de forma independiente de la resolución. La dirección de cruce depende del modo elegido, ya que en horizontal se selecciona entre izquierda a derecha o derecha a izquierda, mientras que en vertical se selecciona entre arriba a abajo o abajo a arriba.



Modo de conteo

- Horizontal (línea vertical)
- Vertical (línea horizontal)
- Horizontal (línea vertical) ✓

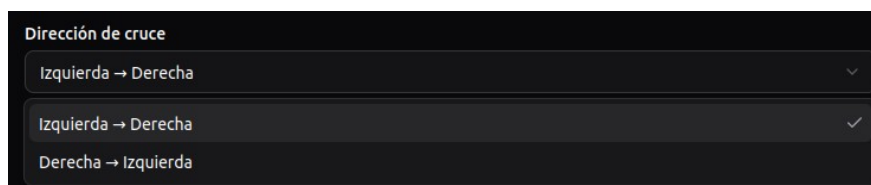
(a)



Línea de cruce (X normalizada, 0-1)

0.5

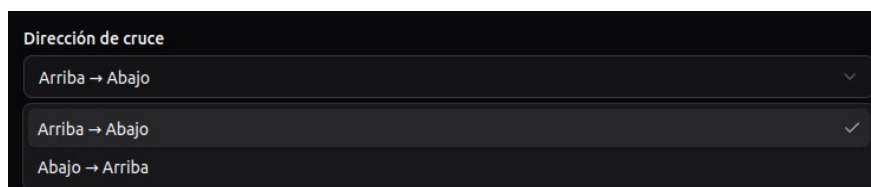
(b)



Dirección de cruce

- Izquierda → Derecha
- Izquierda → Derecha ✓
- Derecha → Izquierda

(c)



Dirección de cruce

- Arriba → Abajo
- Arriba → Abajo ✓
- Abajo → Arriba

(d)

FIGURA 31. Selectores de la configuración del conteo desplegados: (a) modo de conteo; (b) posición de la línea de cruce normalizada; (c) dirección de cruce en modo horizontal; (d) dirección de cruce en modo vertical.

2.8. Sincronización entre el servidor y el robot

- Ofrece el botón "Sincronizar ahora", que fuerza de inmediato el envío de sesiones, eventos y grabaciones al servidor central sin esperar al ciclo automático periódico (ver FIGURA 32).



FIGURA 32. Sección de sincronización con el botón "Sincronizar ahora", que fuerza una sincronización inmediata con el servidor central.

2.9. Configuración del servidor central

- Define la URL y las credenciales con que el robot se conecta al servidor central, a través de cuatro campos:
 - En la Server URL se escribe la dirección donde responde el servidor central, de la forma `http://<host>:<puerto>`, que el robot usa como destino de la sincronización.
 - En el Device ID se indica el identificador con el que ese robot queda registrado en el servidor, de modo que sus sesiones y grabaciones se asocien al dispositivo correcto.
 - En la API Key se ingresa la clave de dispositivo que el servidor entregó al robot, y con la que este se autentica en cada sincronización.
 - En la URL LAN de video se puede configurar la dirección del servidor dentro de la red local, para que las grabaciones se suban por la LAN, que es más rápida, cuando esa red está disponible; si se deja vacía, las grabaciones se suben por internet.
- El robot sincroniza de forma automática y periódica, pero solo cuando el servidor central está alcanzable; si no responde, omite el ciclo y reintenta más tarde (ver FIGURA 33).

Configuración

Cámara
Dispositivo y resolución

Detección
Qué detectar y cómo

Conteo
Línea de cruce y dirección

Sincronización
Forzar sync ahora

Servidor
Conexión y credenciales

Servidor
URL y credenciales del servidor central

Server URL

Device ID

API Key

Guardar

URL LAN (video)

Dirección del servidor en la red local. Si esta seteada y alcanzable, las grabaciones se suben por LAN (mas rapido) en vez de por internet. Dejar vacío para subir siempre por internet.

Guardar

FIGURA 33. Panel de configuración del servidor central con la Server URL, el Device ID y la API Key, más la URL LAN para subir las grabaciones por la red local.

2.10. Módulo de sesiones

- Lista las sesiones de conteo en una tabla con la fecha, la clase, el conteo, la duración, el tamaño del video y el estado de subida, paginada de a trece entradas (ver FIGURA 34).
- Permite filtrar por clase y por rango de fechas (desde y hasta).
- Cada fila ofrece, de izquierda a derecha, cinco acciones:
 - Sincronizar la sesión al servidor central (icono de nube).
 - Re-procesar el conteo del video con el *counting worker* (icono de recarga).
 - Reproducir el video con las detecciones superpuestas (icono de play).
 - Descargar el video MP4 (icono de descarga).
 - Eliminar la sesión (icono de papelera).
- La columna de conteo muestra el total final, un indicador de "procesando" mientras el *counting worker* termina, o "error" si el conteo falla.

Sesiones

Clase: Todos Desde: Hasta:

Fecha	Clase	Conteo	Duración	Tamaño	Estado	Acciones
30 may 2026, 22:26	humano	0	4s	714 KB	subido	
30 may 2026, 22:24	humano	1	6s	382 KB	subido	
30 may 2026, 22:08	humano	1	7s	604 KB	subido	

FIGURA 34. Módulo de sesiones con la tabla de sesiones, los filtros por clase y fecha y las acciones por fila.

2.11. Módulo de grabaciones

- Lista las grabaciones sin conteo asociado en una tabla con la fecha, la duración, el tamaño y el estado de subida, paginada de a trece entradas (ver FIGURA 35).
- Permite filtrar por rango de fechas (desde y hasta).
- Cada fila ofrece, de izquierda a derecha, cuatro acciones:
 - Sincronizar la grabación al servidor central (icono de nube).
 - Reproducir el video (icono de play).
 - Descargar el video MP4 (icono de descarga).
 - Eliminar la grabación (icono de papelera).
- El estado de subida (grabando, pendiente, subiendo o subido) se actualiza por sondeo, de modo que la grabación recién terminada aparece como "pendiente" hasta que se sincroniza.

Grabaciones (3)

Desde

mm / dd / yyyy

Hasta

mm / dd / yyyy

Fecha	Duración	Tamaño	Estado	Acciones
30 may 2026, 22:25	4s	714 KB	subiendo	   
30 may 2026, 22:24	6s	382 KB	subiendo	   
30 may 2026, 22:07	7s	404 KB	subiendo	   

FIGURA 35. Módulo de grabaciones con la tabla de grabaciones, los filtros por fecha y las acciones por fila.

CAPÍTULO 3

EVALUACIÓN DE LA ESTRATEGIA DE DETECCIÓN Y CONTEO

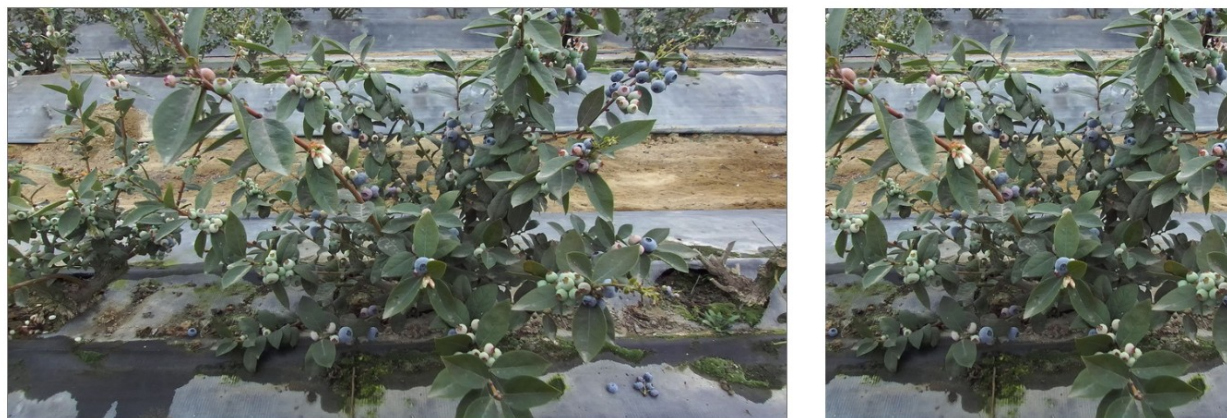
Este capítulo evalúa la estrategia de detección y conteo de frutos de extremo a extremo sobre videos de campo, midiendo el error de conteo (MAE) en lugar de la calidad de detección por *frame* (mAP). El *benchmark* compara dos estrategias de conteo y veinte detectores YOLO (cinco familias por cuatro backbones) sobre grabaciones tomadas en el fundo Danper, y se basa en el estudio de Cubas y Prado (2026). Se reportan la estrategia de conteo propuesta, los modelos de detección evaluados, el protocolo de comparación, los resultados y el estado de la integración del modelo de producción.

3.1 Estrategia de conteo propuesta

La estrategia toma como entrada un video capturado en condiciones de campo y produce como salida el número de frutos contados, mediante cuatro etapas encadenadas: recorte, detección, seguimiento y criterio de conteo.

3.1.1 Recorte

El recorte extrae una región cuadrada centrada de 1080x1080 píxeles del *frame* original de 1920x1080 y la reescala a la entrada de YOLO de 640x640, como muestra la Figura 13. Este paso evita el *letterboxing*, que reduce el tamaño aparente de los frutos, y la distorsión directa, que altera su forma circular.



(a)

(b)

FIGURA 13. Operación de recorte: (a) *frame* original de 1920x1080; (b) región cuadrada centrada de 1080x1080 extraída de él.

3.1.2 Detección

La detección procesa la región recortada con un detector YOLO entrenado para localizar frutos y entrega una lista de cuadros delimitadores $B=(x, y, w, h)$ en píxeles, uno por fruto detectado, como ilustra la Figura 14.



FIGURA 14. Detección de frutos con un modelo YOLO entrenado: (a) frame recortado y reescalado de entrada; (b) el mismo frame con los cuadros delimitadores devueltos por el detector superpuestos.

3.1.3 Seguimiento

El seguimiento asigna a cada fruto un identificador persistente (ID) y lo mantiene consistente entre *frames* consecutivos mediante BoT-SORT (Aharon et al., 2022) con sus parámetros por defecto, necesario porque el movimiento de la cámara hace que un mismo fruto aparezca con coordenadas distintas en *frames* sucesivos. Tras el seguimiento, cada fruto queda representado como un objeto $O = (B, ID)$, que es la unidad de trabajo del criterio de conteo.

3.1.4 Criterio de conteo

El criterio decide, *frame* a *frame*, cuándo un ID entra o sale del conteo según cruce una línea de referencia vertical L que divide el *frame* en una zona de detección ($O_x \leq L$) y una zona de conteo ($O_x > L$), donde $O_x(n)$ es la coordenada horizontal del centro del cuadro de un objeto en el *frame* n . La Figura 15 muestra el esquema. El objeto entra al conteo al cruzar de la zona de detección a la de conteo:

$$O_x(n) > L \text{ y } O_x(n-1) \leq L$$

y sale del conteo al cruzar de vuelta:

$$O_x(n-1) > L \text{ y } O_x(n) \leq L$$

El conteo final es el número de IDs únicos que permanecen en la lista al terminar el video.

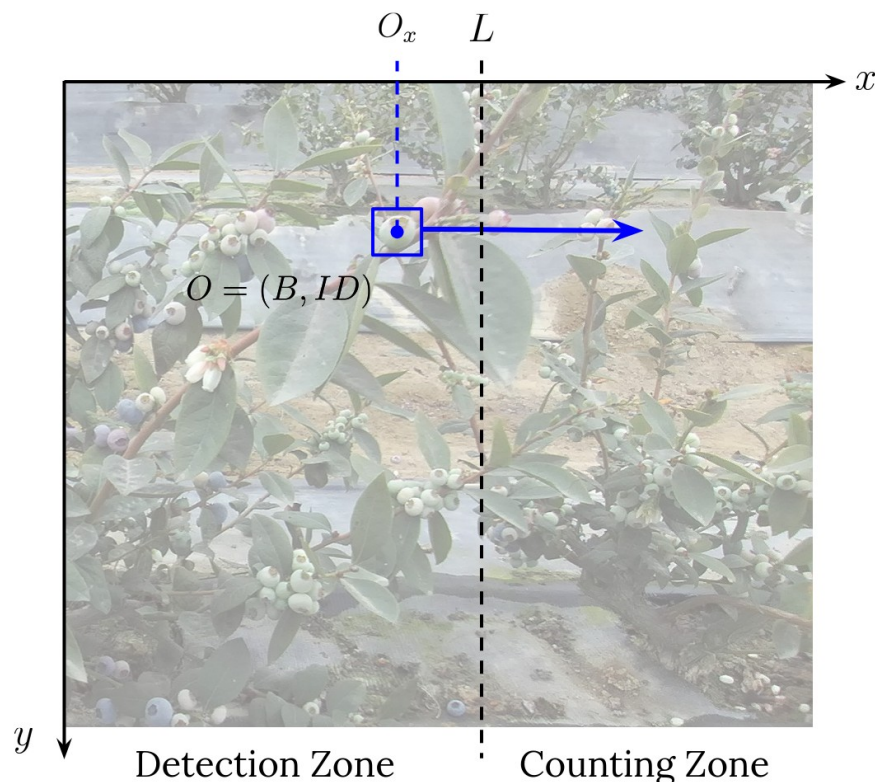


FIGURA 15. Esquema del criterio de conteo con la línea de referencia L que divide las zonas de detección y de conteo. Se ilustra la variante `line_crossing`; `tiled_crossing` replica el mismo esquema de forma independiente en cada uno de los dos mosaicos horizontales, con su propia línea de referencia L_i .

3.1.5 Estrategias de conteo

Se consideran dos estrategias que aplican este criterio:

- **line_crossing:** una sola línea de referencia L ubicada en el centro horizontal del *frame* completo, con un único tracker sobre todas las detecciones; el conteo es el número de IDs únicos que cruzan L .
- **tiled_crossing:** la banda central del *frame* se divide en dos mosaicos horizontales de igual ancho, el mismo criterio se aplica de forma independiente a cada mosaico con su propia línea de referencia L_i y su propia instancia de tracker, y el conteo del video es la suma de los conteos por mosaico, lo que reduce los cambios de identidad en escenas densas al limitar la extensión espacial que cada tracker debe asociar.

3.2 Modelos de detección evaluados

Se evalúan veinte detectores contruidos como el producto de cinco familias YOLO y cuatro backbones por familia. Los cuatro backbones corresponden, en orden creciente de parámetros y capacidad, a las variantes nano/tiny, small, medium y large/compact, según los códigos oficiales de cada familia; por eso YOLOv9 usa las variantes tiny y compact donde las demás familias usan nano y large. La Tabla 9 resume el origen y la contribución técnica de cada familia.

Modelo	Origen	Contribución técnica
YOLOv8	Jocher et al. (2023), Ultralytics	Diseño anchor-free con cabeza de detección desacoplada en el ecosistema Ultralytics.
YOLOv9	Wang et al. (2024), Academia Sinica	Introduce Programmable Gradient Information y la arquitectura RepNCSPPELAN para preservar información gradiente en redes profundas.
YOLOv10	Wang et al. (2024), Tsinghua University	Elimina la operación NMS mediante un esquema de asignación dual one-to-many y one-to-one durante el entrenamiento.
YOLOv11	Ultralytics (2024)	Introduce el bloque C3k2 en el backbone y el bloque C2PSA con atención posicional.
YOLO26	Ultralytics (2025)	Versión más reciente de la familia, orientada a inferencia eficiente en el borde.

TABLA 9. Familias de detección evaluadas en el benchmark de conteo.

Los veinte detectores se entrenan desde cero bajo condiciones idénticas. La Tabla 10 resume la configuración de entrenamiento.

Item	Valor
Dataset de detección	800 imágenes de arándanos anotadas manualmente, partición 561/160/78 (70/20/10 en train/validación/test)
Framework	Ultralytics
Épocas	50 (early stopping con paciencia 20)
Tamaño de batch	16
Tamaño de imagen	640x640
Hardware	GPU NVIDIA T4
Precisión numérica	mixta automática (AMP)
Aumentación de datos	override propio sobre los valores por defecto de Ultralytics

TABLA 10. Configuración de entrenamiento compartida por los veinte detectores.

El *dataset* de detección consta de 800 imágenes no públicas recolectadas en los campos del fundo Danper y anotadas manualmente en la plataforma Roboflow, y se usa exclusivamente para entrenar los detectores.

3.3 Protocolo de comparación

Los parámetros del pipeline se clasifican en variables, que definen los ejes de comparación, y fijos, que toman un único valor para aislar el efecto de los ejes variables. La Tabla 11 los lista.

Parámetro	Rol	Valor(es)
Estrategia de conteo	variable	line_crossing, tiled_crossing
Pesos del detector YOLO	variable	20 detectores (5 familias x 4 backbones)
Umbral de confianza	fijo	0,15
Algoritmo de tracking	fijo	BoT-SORT
ReID del tracker	fijo	desactivado
Resolución de entrada	fijo	1080x1080 (recorte centrado) reescalada a 640x640
Posición de la línea	fijo	vertical, centrada

TABLA 11. Parámetros del protocolo de evaluación.

1. **Rejilla de configuraciones:** 40 configuraciones, el producto de los dos ejes variables (dos estrategias por veinte detectores). Cada una se ejecuta de extremo a extremo sobre cada video y produce un conteo automático C_v , igual al número de IDs únicos que cruzan la línea de referencia (la suma sobre las dos líneas de los mosaicos en tiled_crossing).
2. **Referencia:** cada video tiene un conteo manual GT_v obtenido por un anotador experto sobre la misma línea.
3. **Dataset de conteo:** cinco clips de 3 segundos recortados de las grabaciones de surco completo, cada uno con su conteo manual; a 20 FPS el presupuesto de *frames* sostiene el seguimiento. La detección y el conteo provienen de campañas distintas en el mismo fondo, por lo que el MAE mide la generalización a una distribución relacionada pero no idéntica.
4. **Intervalo de confianza:** el IC95% se obtiene por bootstrap no paramétrico de 10000 iteraciones con semilla fija, cuyos percentiles 2,5 y 97,5 definen el intervalo ($V=5$).
5. **Métricas:** MAE y sesgo, promediados sobre los V videos.

Ambas métricas promedian el error relativo de conteo sobre los V videos, con C_v el conteo automático y GT_v la referencia manual. El MAE toma el valor absoluto, mientras que el sesgo conserva el signo, negativo si el sistema subcuenta y positivo si sobrecuenta.

$$MAE = \frac{1}{V} \sum_v \frac{|C_v - GT_v|}{GT_v}$$

$$\text{Sesgo} = \frac{1}{V} \sum_v \frac{C_v - GT_v}{GT_v}$$

3.4 Resultados

El protocolo de evaluación de la sección 3.3 se aplica a los cinco videos del conjunto de evaluación. Los resultados se organizan en tres partes: la comparación principal entre las 40 configuraciones y dos barridos sobre los parámetros fijos del protocolo (umbral de confianza y ReID del tracker), ambos sobre la configuración de referencia tiled_crossing + YOLOv9s.

3.4.1 Comparación de estrategias y detectores

La Figura 16 compara el MAE de conteo de cada detector bajo las dos estrategias, con el umbral de confianza y el tracker fijados según la Tabla 11.

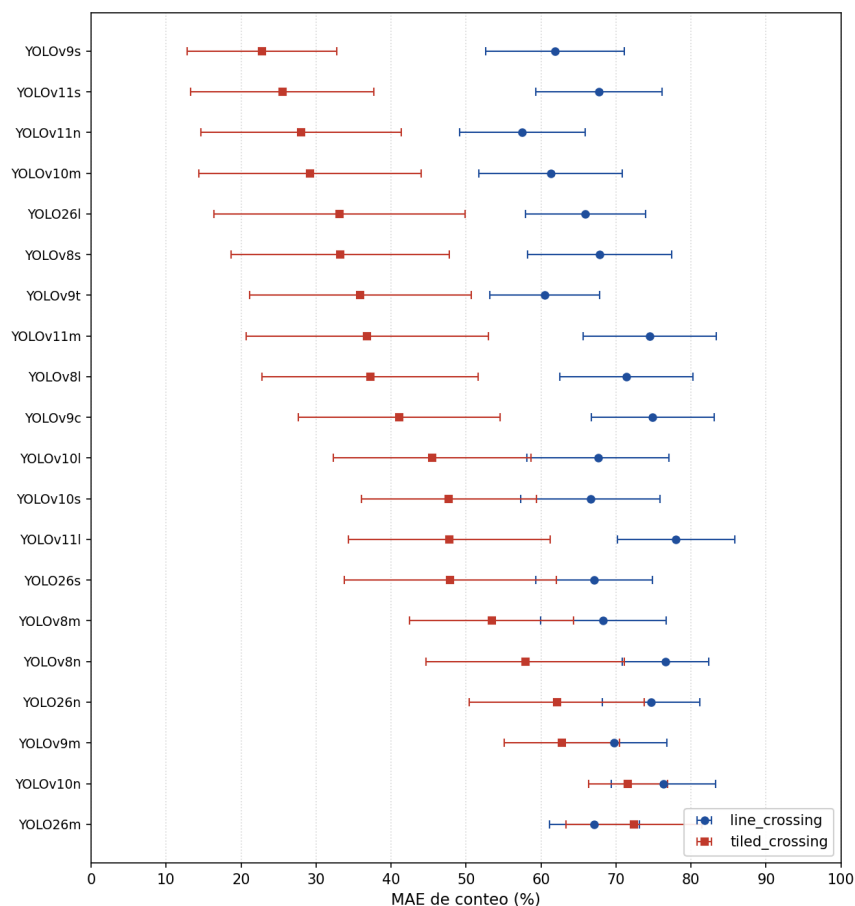


FIGURA 16. MAE de conteo por detector bajo line_crossing (azul) y tiled_crossing (rojo); las barras indican el intervalo de confianza al 95% por bootstrap. Los detectores se ordenan por el MAE de tiled_crossing, con YOLOv9s (menor MAE) en la parte superior.

`tilted_crossing` reduce el MAE de conteo respecto de `line_crossing` en 19 de los 20 detectores; la única excepción es YOLO26m, donde `tilted_crossing` resulta 5,3 puntos peor. El menor MAE de toda la rejilla es 22,8% (IC95% [12,8, 32,7]; sesgo -13,8%, IC95% [-30,3, +4,4]), alcanzado por `tilted_crossing` combinado con YOLOv9s. Aunque el IC del sesgo de esta configuración cruza el cero, las estimaciones puntuales son negativas en los 20 pares detector-estrategia, consistente con el techo del detector, que en escenas densas localiza alrededor de la mitad de los arándanos presentes en el *frame*. Con $V=5$ varias configuraciones de bajo MAE son estadísticamente indistinguibles; por ejemplo, YOLOv9s (22,8%) y YOLOv11s (25,5%, IC95% [13,1, 37,5]) solapan sus intervalos, y estrecharlos requiere anotar videos de evaluación adicionales.

3.4.2 Sensibilidad al umbral de confianza

La Tabla 12 reporta el MAE y el sesgo de la configuración de referencia `tilted_crossing` + YOLOv9s para seis valores del umbral de confianza, entre 0,05 y 0,40.

Umbral de confianza	MAE (%)	Sesgo (%)
0,05	22,8	-13,8
0,10	22,8	-13,8
0,15	22,8	-13,8
0,20	22,8	-13,8
0,30	23,2	-15,4
0,40	24,3	-20,3

TABLA 12. Sensibilidad del MAE y el sesgo de `'tilted_crossing'` + YOLOv9s al umbral de confianza.

El MAE y el sesgo son constantes entre 0,05 y 0,20, en 22,8% y -13,8% respectivamente, y solo empiezan a crecer a partir de 0,30. El umbral de 0,15 fijado en la Tabla 11 queda por tanto dentro del rango estable de operación y no sesga la comparación hacia ningún detector en particular.

3.4.3 Efecto del ReID en el tracker

La Tabla 13 compara la configuración de referencia `tilted_crossing` + YOLOv9s con el tracker BoT-SORT configurado para usar las características de apariencia por defecto del backbone del detector (`with_reid = true`, sin modelo de ReID dedicado ni ajuste).

Configuración	MAE (%)
ReID desactivado	22,8
ReID activado	21,6

TABLA 13. Efecto del ReID en `'tilted_crossing'` + YOLOv9s.

Activar el ReID reduce el MAE de 22,8% a 21,6%, un cambio de 1,2 puntos que queda por debajo de la variabilidad observada entre detectores en la Figura 16. La contribución de la

aparición a la asociación es limitada porque los arándanos son pequeños y visualmente similares entre sí, de modo que el tracker no es el cuello de botella del pipeline.

CAPÍTULO 4

CLASIFICACIÓN DE FRUTOS POR NIVELES DE MADUREZ

Este capítulo compara cómo distintos paradigmas de representación capturan la madurez del arándano a partir de imágenes individuales del fruto segmentado. El pipeline de detección y conteo del capítulo 3 localiza al fruto en el *frame*, mientras que el clasificador asigna a cada fruto un nivel de madurez, lo que habilita reportes por estado fenológico además del conteo agregado por camellón. El capítulo se basa en el estudio de clasificación de madurez de arándanos por aprendizaje de representaciones (Cubas, 2026b). Se reportan el problema y el enfoque, el *dataset*, los paradigmas evaluados, el protocolo de evaluación y los resultados.

4.1 Definición del problema y enfoque

La madurez del arándano es un fenómeno cromático continuo, con una transición gradual de verde a azul, pero las etiquetas disponibles son discretas e inciertas porque las clases se solapan en los rangos de color intermedios. A esto se suman dos dificultades del régimen de trabajo: imágenes pequeñas (recortes del fruto de entre 100 y 200 píxeles estandarizados a 128x128) y degradación por ruido de iluminación y baja resolución.

Trabajos previos de clasificación de madurez con redes convolucionales supervisadas reportan alta exactitud en manzanas (Zhang et al., 2018), mangos (Naranjo-Torres et al., 2020) y tomates (Liu et al., 2019), pero tratan la madurez como un conjunto de clases discretas. Esta tensión entre un fenómeno gradual y un etiquetado categórico motiva la pregunta central del estudio, que es si las representaciones aprendidas, en particular las *self-supervised*, capturan la estructura continua de la madurez mejor que la representación interna de un clasificador supervisado, por lo que el enfoque no compara solo la precisión sino la geometría del espacio latente que induce cada paradigma.

4.2 Dataset y preprocesamiento

El *dataset* consta de 1239 imágenes del fruto segmentado (fondo removido sobre blanco) recolectadas en el laboratorio, distribuidas en siete clases de madurez ordenadas por progresión cromática: verde, cremoso, rosado, pintón 1, pintón 2, guinda y azul. Las clases están aproximadamente balanceadas, con alrededor de 180 imágenes por clase (pintón 2 es la única excepción menor, con 159).

El preprocesamiento estandariza cada recorte a 128x128 píxeles por estiramiento y aplica variaciones de brillo y color como *data augmentation*, para evaluar robustez ante iluminación y mitigar el tamaño reducido del conjunto. La partición es 70/15/15 estratificada por clase con semilla fija (867 imágenes de entrenamiento, 186 de validación y 186 de prueba); el conjunto de prueba queda reservado para evaluar los *embeddings* congelados.

4.3 Paradigmas de representación evaluados

Se comparan seis representaciones bajo condiciones equivalentes (misma partición, capacidad de *encoder* comparable, mismo presupuesto de entrenamiento y mismo protocolo de evaluación), todas con arquitecturas CNN pequeñas acordes al tamaño de imagen. La Tabla 14 las resume.

Representación	Paradigma	Latente
----------------	-----------	---------

Supervisado	clasificador CNN de referencia; <i>embedding</i> de la penúltima capa	continuo, 64-d
Autoencoder vanilla	reconstructivo con latente continuo	continuo, 64-d
VQ-VAE	reconstructivo con cuantización vectorial; latente discreto tomado de un <i>codebook</i> finito (Van den Oord et al., 2017)	discreto, 128-d
RVQ-VAE	reconstructivo con cuantización vectorial residual en cascada	discreto-continuo, 128-d
JEPA	predictivo en espacio latente; predice regiones enmascaradas sin reconstruir píxeles (Assran et al., 2023)	continuo, 128-d
Métrica continua	aprendizaje métrico; la distancia en el <i>embedding</i> refleja la similitud de madurez (Schroff et al., 2015)	continuo, 128-d

TABLA 14. Paradigmas de representación evaluados y dimensión del espacio latente.

4.4 Protocolo de evaluación

Cada representación se entrena sin usar las etiquetas en los métodos *self-supervised*, se congela el *encoder* y se evalúa la calidad del *embedding* resultante con dos sondas sobre el conjunto de prueba: una sonda lineal (un clasificador lineal entrenado sobre el *embedding* congelado) y un clasificador k-NN. Se reportan el accuracy y el F1-macro de cada sonda, de modo que la sonda lineal mide la separabilidad global de las clases y el k-NN su estructura local. Para inspeccionar la geometría de cada espacio, los *embeddings* se proyectan a dos dimensiones con t-SNE, una técnica de reducción de dimensionalidad que preserva la vecindad local de los puntos (Van der Maaten y Hinton, 2008).

4.5 Resultados

La Tabla 15 reporta el desempeño de cada representación bajo las dos sondas, ordenado por el accuracy de la sonda lineal.

Representación	Sonda lineal acc. (%)	Sonda lineal F1-macro	k-NN acc. (%)
Supervisado	78,0	0,78	77,4
Métrica continua	69,4	0,68	65,1
RVQ-VAE	51,6	0,50	36,6
Autoencoder vanilla	44,1	0,37	33,9
JEPA	43,5	0,38	23,1

VQ-VAE	38,2	0,35	36,0
--------	------	------	------

TABLA 15. Desempeño de las representaciones por sonda lineal y k -NN sobre el conjunto de prueba (186 imágenes, siete clases).

Hallazgos principales:

- El clasificador supervisado obtiene el mayor accuracy (78,0% en sonda lineal), como es esperable cuando las etiquetas guían directamente la representación.
- Entre los métodos sin supervisión por clase, la representación métrica continua (69,4%) y la RVQ-VAE (51,6%) superan al autoencoder vanilla (44,1%), a JEPA (43,5%) y a la VQ-VAE (38,2%).
- El autoencoder continuo supera a la VQ-VAE discreta en la sonda lineal (44,1% frente a 38,2%), consistente con la hipótesis de que un latente continuo preserva mejor la progresión gradual de la madurez.
- El k -NN cae más que la sonda lineal en los latentes discretos y en JEPA (23,1% en JEPA), lo que indica una geometría latente menos separable a nivel local pese a una separabilidad lineal comparable.

El conjunto de prueba es pequeño (186 imágenes) y varias diferencias caen dentro de la variabilidad esperable, por lo que el eje de interpretabilidad geométrica resulta tan relevante como el accuracy para decidir qué representación captura mejor la trayectoria continua de verde a azul. Las Figuras 17 a 20 muestran los *embeddings* de cuatro representaciones proyectados con t-SNE y coloreados por madurez de verde a azul.

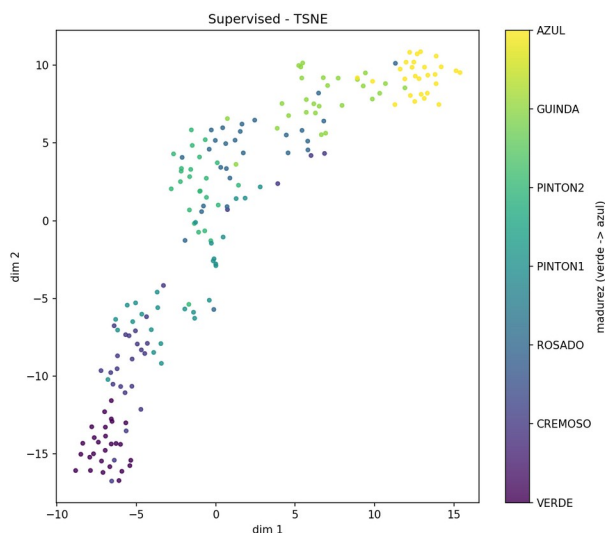


FIGURA 17. Proyección t-SNE de los *embeddings* del clasificador supervisado, coloreados por nivel de madurez.

En la Figura 17 se observa que el supervisado agrupa los frutos en regiones compactas por clase, con la madurez ordenada de extremo a extremo y fronteras nítidas entre niveles.

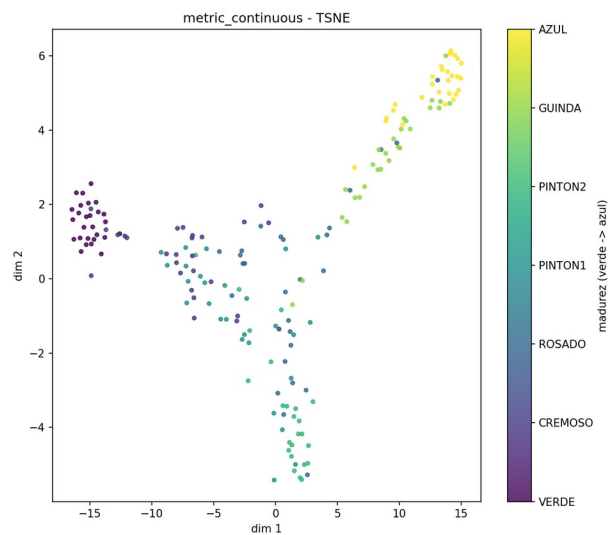


FIGURA 18. Proyección t-SNE de los embeddings de la representación métrica continua, coloreados por nivel de madurez.

En la Figura 18 se observa que la métrica continua traza la trayectoria cromática más ordenada de verde a azul entre las representaciones evaluadas.

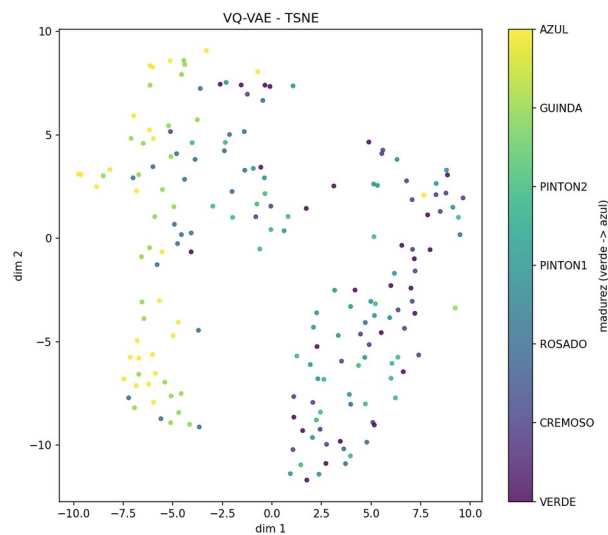


FIGURA 19. Proyección t-SNE de los embeddings de VQ-VAE, coloreados por nivel de madurez.

En la Figura 19 se observa que la VQ-VAE fragmenta el espacio, con los frutos azules separados pero los niveles intermedios mezclados sin una progresión clara.

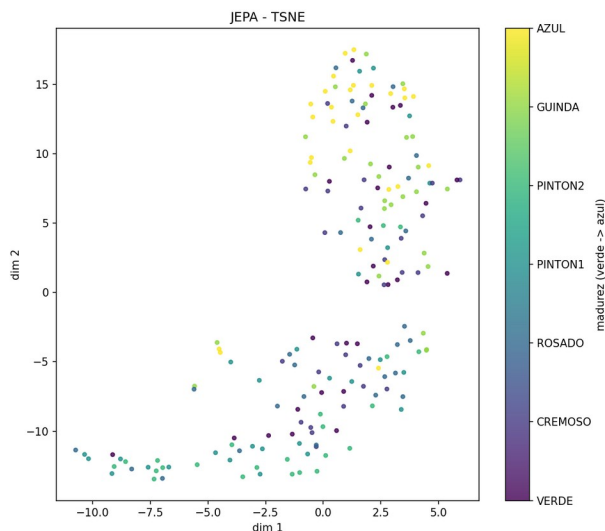


FIGURA 20. Proyección t-SNE de los embeddings de JEPa, coloreados por nivel de madurez.

En la Figura 20 se observa que JEPa presenta una geometría difusa con solapamiento entre niveles, coherente con su menor desempeño en las sondas.

CONCLUSIONES

1. La arquitectura por procesos independientes resuelve los problemas de aislamiento de fallos, desacoplamiento de tasas de *frame* y conflictos de versiones de Python que presentaba el sistema monolítico anterior. Captura, inferencia y grabación operan en simultáneo a 1080p y 30 FPS sin acumulación de buffers ni regresión en el módulo de visión.
2. La aceleración con TensorRT FP16 sobre los Tensor Cores de la Jetson Xavier reduce la latencia de inferencia aislada de 75 ms a 50,9 ms en el percentil 50 y eleva el FPS efectivo medido de extremo a extremo de 9 a 14, manteniendo el modelo intacto sin alterar la métrica de detección.
3. El *benchmark* de conteo sobre videos de campo compara dos estrategias y veinte detectores (cinco familias por cuatro backbones). La configuración *tilled_crossing* + YOLOv9s obtiene el menor error de conteo (MAE 22,8%, sesgo -13,8%) y *tilled_crossing* reduce el MAE respecto de *line_crossing* en 19 de los 20 detectores. El sesgo negativo en todas las configuraciones identifica al detector como la fuente dominante de error, y YOLOv9s queda seleccionado como modelo de producción para la detección de frutos.
4. La plataforma carga YOLOv11 preentrenado como modelo de validación integral, lo que permite verificar el sistema sobre el robot real sin depender de la disponibilidad estacional de arándanos en campo, mientras YOLOv9s se publica como *checkpoint* productivo a través del mecanismo de distribución ya operativo.
5. El servidor central quedó desplegado en la PC del laboratorio mediante Docker Compose, con acceso remoto por Tailscale Funnel sobre HTTPS y endurecimiento de

autenticación (límite de tasa, bloqueo de cuenta, CORS restringido y cabeceras de seguridad), lo que habilita la sincronización de datos y la distribución de modelos hacia el robot. La plataforma incorpora además el soporte de cámara IP por RTSP, el transporte de video por WebCodecs sobre WebSocket y el registro de detecciones por *frame* que permite reproducir cada sesión grabada con las detecciones superpuestas.

6. El estudio de clasificación de madurez compara seis paradigmas de representación sobre un *dataset* de 1239 imágenes de arándanos segmentados en siete clases ordenadas por progresión cromática, evaluados por sondas lineal y k-NN sobre los *embeddings* congelados. El clasificador supervisado obtiene el mayor accuracy (78,0% en sonda lineal), seguido por la representación métrica continua (69,4%) y la RVQ-VAE (51,6%); el autoencoder continuo supera a la VQ-VAE discreta, consistente con la naturaleza gradual de la madurez. Los resultados son preliminares y orientan la integración del *encoder* supervisado detrás del detector YOLOv9s en el robot, junto con el análisis de interpretabilidad de los espacios latentes.
7. Las líneas de trabajo siguientes apuntan a ampliar el *dataset* de entrenamiento del detector para reducir el subconteo en escenas densas y completar el mapa sin conexión para zonas sin red.

REFERENCIAS

Aharon, N., Orfaig, R. y Bobrovsky, B.-Z. (2022). BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv:2206.14651. <https://arxiv.org/abs/2206.14651>

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y. y Ballas, N. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Cubas, P. (2025). Informe técnico #2: Evaluación de algoritmos de detección de objetos para conteo de arándanos. Proyecto PE5010-86701-2024-PROCIENCIA, Universidad Privada Antenor Orrego.

Cubas, P. (2026a). Informe técnico #3: Plataforma de software del robot móvil agrícola, versión inicial. Proyecto PE5010-86701-2024-PROCIENCIA, Universidad Privada Antenor Orrego.

Cubas, P. (2026b). Self-Supervised Representation Learning Applied to Blueberry Ripeness Classification. Laboratorio de Investigación Multidisciplinario (LABINM), Universidad Privada Antenor Orrego.

Cubas, P. y Prado, S. (2026). A Detection-and-Tracking Pipeline for Fruit Counting: Benchmarking the Modern YOLO Family (v8 to YOLO26) on Pre-Harvest Blueberries. Laboratorio de Investigación Multidisciplinario (LABINM), Universidad Privada Antenor Orrego.

He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep Residual Learning for Image Recognition. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Jocher, G., Chaurasia, A. y Qiu, J. (2023). Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>

- Liu, Z. et al. (2019). Tomato diseases and pests detection based on improved YOLO v3 convolutional neural network. *Frontiers in Plant Science*.
- Naranjo-Torres, J. et al. (2020). A review of convolutional neural network applied to fruit image processing. *Applied Sciences*, 10(10), 3443.
- Schroff, F., Kalenichenko, D. y Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Simonyan, K. y Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR 2015*.
- Ultralytics. (2024). YOLO11: Documentation and release notes. <https://docs.ultralytics.com/models/yolo11/>
- Ultralytics. (2025). YOLO26: Documentation and release notes. <https://docs.ultralytics.com/models/yolo26/>
- Van den Oord, A., Vinyals, O. y Kavukcuoglu, K. (2017). Neural Discrete Representation Learning (VQ-VAE). *Advances in Neural Information Processing Systems (NeurIPS)*.
- Van der Maaten, L. y Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579-2605.
- Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J. y Ding, G. (2024). YOLOv10: Real-Time End-to-End Object Detection. *arXiv:2405.14458*. <https://arxiv.org/abs/2405.14458>
- Wang, C.-Y., Yeh, I.-H. y Liao, H.-Y. M. (2024). YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. *arXiv:2402.13616*. <https://arxiv.org/abs/2402.13616>
- Zhang, Y. et al. (2018). Identification of apple leaf diseases based on deep convolutional neural networks. *Symmetry*, 10(1), 11.